Hack alert

Two security experts recently took control of a Windows 7 system during the boot process using a code they developed. Reassuring to know it can't be done remotely.

XP Popularity

According to Forrester research, Windows XP powers 71 per cent of all PCs

There are a multitude of options available at this point. We could use 'name' instead of length to obtain the file name and test it however we like, or the modification date, or the attributes, the possibilities are endless. Even while considering the file size, we can change the '-gt' to '-lt' to get all files smaller that 50 kB or '-eq' to get all files of exactly 50 kB.

This command will test each file object for the prescribed condition, and output those that comply. This output is now further piped to:

```
Rename-Item -newname {$_.name + '.big'}
```

This final part of the command gets the filtered list of files from the previous command, and renames them according to the parameters specified. The '-newname' parameter, as one might guess, specifies the new name to give to each file. Here we simply take the input file name '$_.name', and add a '.big' after it. Simple!

However we can do much, much more, by using the '-replace' parameter for example, we can replace certain strings in the file name by other strings. For example, we could replace all instances of 'Music' with 'Song' by:

```
Rename-Item -newname {$_.name -replace 'Music','Song'}
```

PowerShell to the Rescue

So we renamed a couple of file. What more can we do?

PowerShell is extremely diverse. It supports functions, loops, if-else conditions, reading / writing to files, downloading files from the internet, administering windows settings, regular expressions, pipelining, etc. In fact, all the cmdlets are actually .NET classes, so you can create some yourself.

Let's take a look at some of the things you can easily do with PowerShell.

Write the list of files in a directory to an HTML formatted file

```
Get-ChildItem | ConvertTo-Html | Out-File .\dirlist.htm
```

Here we use the 'ConvertTo-HTML' cmdlet to get an HTML output, and use the 'Out-File' cmdlet to write it to a file. We could as easily get a CSV output for use in a spreadsheet by using the 'ConvertTo-CSV' cmd.

Organise files in folder by name

```
$filelist = Get-ChildItem
$filelist | ForEach-Object -Process {mkdir $_.name.substring(0,1) -ErrorAction SilentlyContinue }
$filelist | Move-Item -Destination {$_.name.substring(0,1)}
```

Now here's some actual scripting work. This is what's going on:

First we store the list of files in the current folder to a variable called '$filelist'.

Secondly we use the 'ForEach-Object' cmdlet to perform an operation on each file on the folder. The '-Process' parameter tells PowerShell the action to be performed for each object. What we are doing is creating a new folder for each file. The folder name is the same as the first character of the file name (which we get using $_.name.SubString(0,1) which extracts 1 character starting from the 0th position of the filename). This will essentially create folders named '0', '1','2'...'A','B','C'... depending on the names of the files in current folder. Here, we have also added a parameter '-ErrorAction SilentlyContinue' which tells the shell not to display any errors that occur. This is done as there will be many error messages displayed since many files will begin with the same name.

Finally, the last command takes our file list, and moves each file to a folder corresponding to the first character in its name.

List all processes which are taking up to much memory

```
Get-Process | Where-Object {$_.VM -gt 500MB}
```

Simple enough, this takes the output of Get-Process and pipes it through the 'Where-Object' cmdlet to get a list of processes which have a 'VM' (Virtual Memory usage) greater than 500 MB. You can further pipe this to the 'Stop-Process' cmdlet to stop all such processes!

List all large files in a drive

```
Get-ChildItem -Recurse | Where-Object {$_.Length -gt 300MB} | Sort-Object -Property length -Descending
```

Here we take the recursive directory listing of the drive using 'Get-ChildItem', and pipe it to 'Where-Object' for filtering. 'Where-Object' then filters out the files greater than 300 MB, and pipes them to 'Sort-Object'. 'Sort-Object', as the name suggests, is used for sorting the data. Here we sort it based on the length (i.e. size) of the file, and also tell the 'Sort-Object' to arrange them in descending order. You can use a multiple order sort by specifying multiple properties separated by commas e.g. '-Property length, name'. This information can be very useful when cleaning up your hard drive. So you can, if you want, further pass this data to the 'ConvertTo-HTML' or ConvertTo-CSV' cmdlets followed by the 'Out-File' cmdlet to generate a report of sorts. For example,

```
Get-ChildItem -Recurse | Where-Object {$_.Length -gt 300MB} | Sort-Object -Property length -De | ConvertTo-Csv | Out-File report.csv
```

Empower the PowerShell!

You thought that was all!?

Microsoft and many third party developers have created PowerShell extensions, which add cmdlets or other features to control many more aspects of the computer. Till windows Vista, PowerShell was an add-on product that needed to be installed, however starting with Windows 7 onwards, it will come integrated with Windows. Microsoft will also drop support for the old Windows Scripting Host, and focus on PowerShell instead.

PowerShell v2, which is currently in development, is now available as a pre-release version. It will feature some powerful new features, and even a GUI based script editor called PowerShell ISE (Integrated Scripting Environment). Some of its interesting features being:

- **Remoting:** allowing you administer a remote computer
- **Background Jobs:** enables you to run commands in the background, asynchronously, without the need of the console.
- **Modules:** allow for modular programming of PowerShell scripts, by compartmentalising your code. It adds a file type '.psm1' for module files
- **Script Cmdlets:** these allow you to make cmdlets using PowerScript itself! No programming knowledge required.
- **And many other features such as:** Transactions, Eventing, Exception Handling, Steppable Pipelines, Internationalisation, Debugging etc.

Best of all, PowerShell is a free tool for Windows users, so it's worth getting even if you use it only occasionally. A large number of ready-made scripts are already available online, from Microsoft and others to ease administrative tasks, so even if you don't want to commit to learning it, it can still ease a lot of your tasks. With PowerShell becoming a core part of Windows from Windows 7 onwards, an elementary knowledge of it will surely go a long way. ■

readersletters@thinkdigit.com